The **`sendNow`** method sends notifications immediately, even if they implement the **`ShouldQueue`** interface.

- Notifications can be delayed using the delay method, allowing you to set specific times for notification delivery. You can set different delay times for different channels as well.
- You can choose a specific queue connection for notifications by using the **`onConnection`** function. Additionally, you can define the queue connection for each notification channel using the **`viaConnections`** method.
- You can specify the queue for each notification channel using the **`viaQueues`** method. Be aware of potential conflicts between database transactions and queued notifications.
- Using the route method from the message facade, you can send notifications to recipients not registered as users in your application, providing ad-hoc notification routing information.
- Customize the email components using the **`toMail`** function, allowing you to define the structure of the email content. You can also customize the sender's address and utilize custom email templates for rendering notifications.
- Leverage the view method to define custom templates for rendering notification emails, enabling the creation of personalized and visually appealing email notifications.
- Use the **`attachData`** and attach methods to include attachments or raw data in email notifications, enhancing the informational value and interactivity of the emails.
- Implement tags and metadata in the email message to organize and monitor the emails sent by the application, providing additional context and information for tracking and management purposes.
- Use the **`withSymfonyMessage`** function to fully customize the message before sending it, allowing for more granular control over the content and structure of the email message.
- Implement the **`toBroadcast`** function to send notifications over Laravel's event broadcasting services, enabling real-time communication with JavaScript-powered frontends.